CASO DE ESTUDIO 2 / ENTREGABLE 01

Arquitectura de datos

Sistema de recompensas dinámico de MiMcDonald's

Diagrama del sistema y de la prueba de concepto

Presentamos la arquitectura inferida del programa y la organización local del pipeline analítico. Distinguimos las hipótesis de infraestructura de los componentes implementados con datos sintéticos.

Integrante	Carné
Nelson García Bravatti	22434
Ricardo Chuy	221007
Diego Valenzuela	22309
Daniel Dubón	22233
Sergio Orellana	221122
Joaquín Puente	22296

Contenido

01 Arquitectura inferida del programa

02 Infraestructura local de la POC

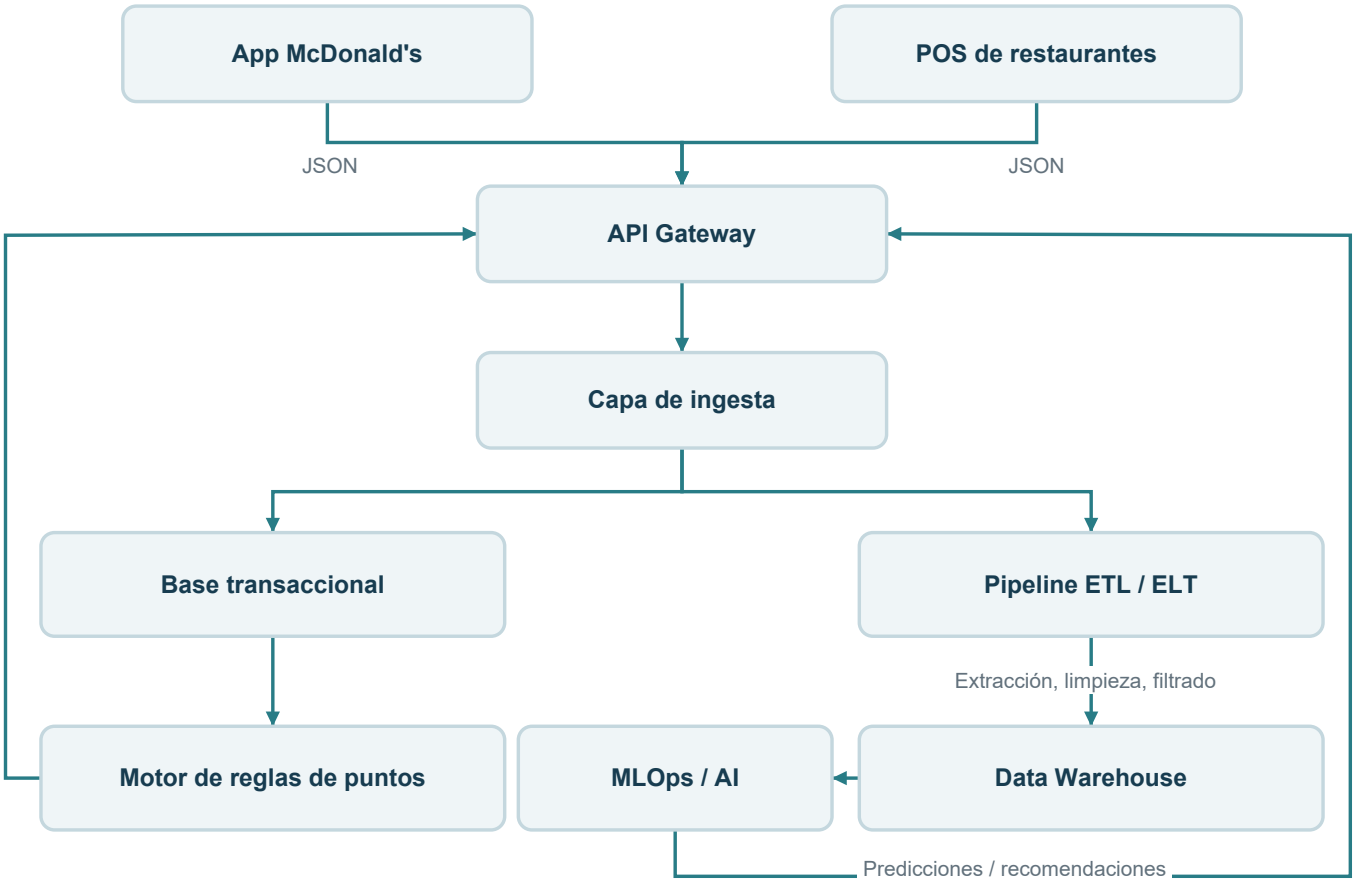
03 Flujo de datos Medallion

Repositorio de GitHub

Machine Learning Engineering · Universidad del Valle de Guatemala

1 Arquitectura inferida a partir del funcionamiento del programa

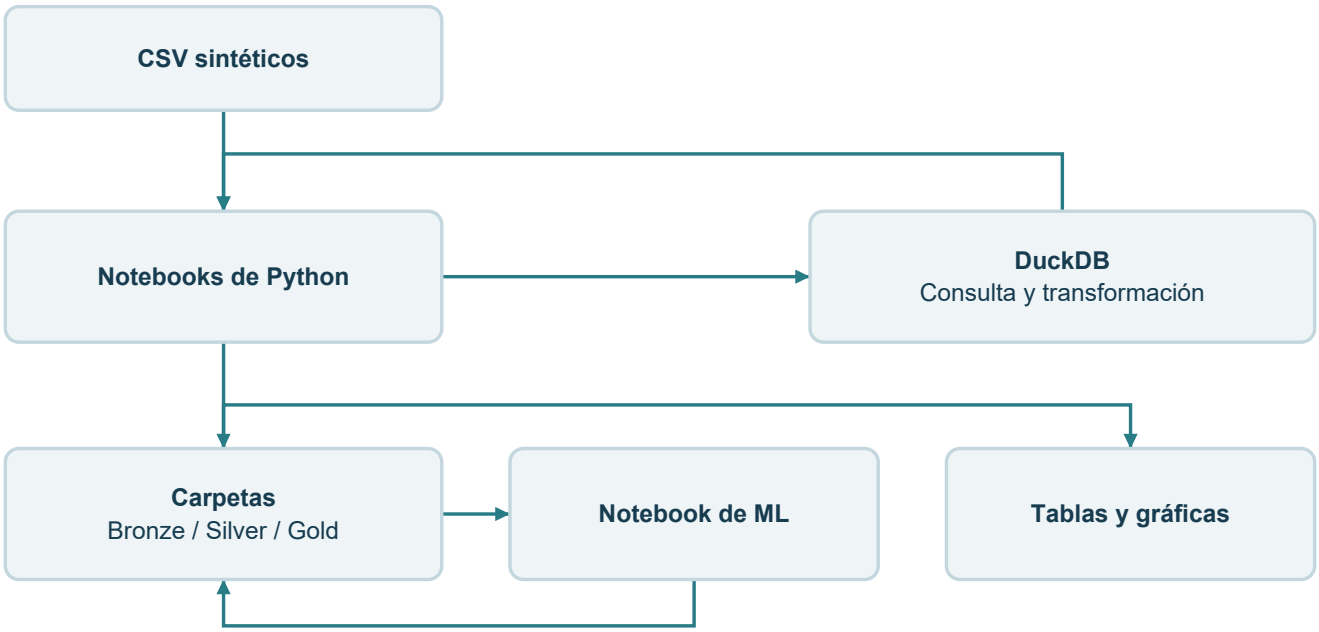
A partir del funcionamiento público del programa, planteamos el siguiente esquema de arquitectura. En este diseño, las compras de la app y de los restaurantes llegan a una API central y alimentan una base transaccional y un pipeline analítico. La presencia de API Gateway, Data Warehouse y ML es una hipótesis de diseño, no una descripción verificada de los sistemas de McDonald's.



Nuestra prueba de concepto se enfoca en la **parte analítica** de esta arquitectura: el pipeline, el Data Warehouse y la capa de ML. Las fuentes de datos (app y POS) se simulan con archivos CSV, para representar sus transacciones.

2 Arquitectura de infraestructura de la POC

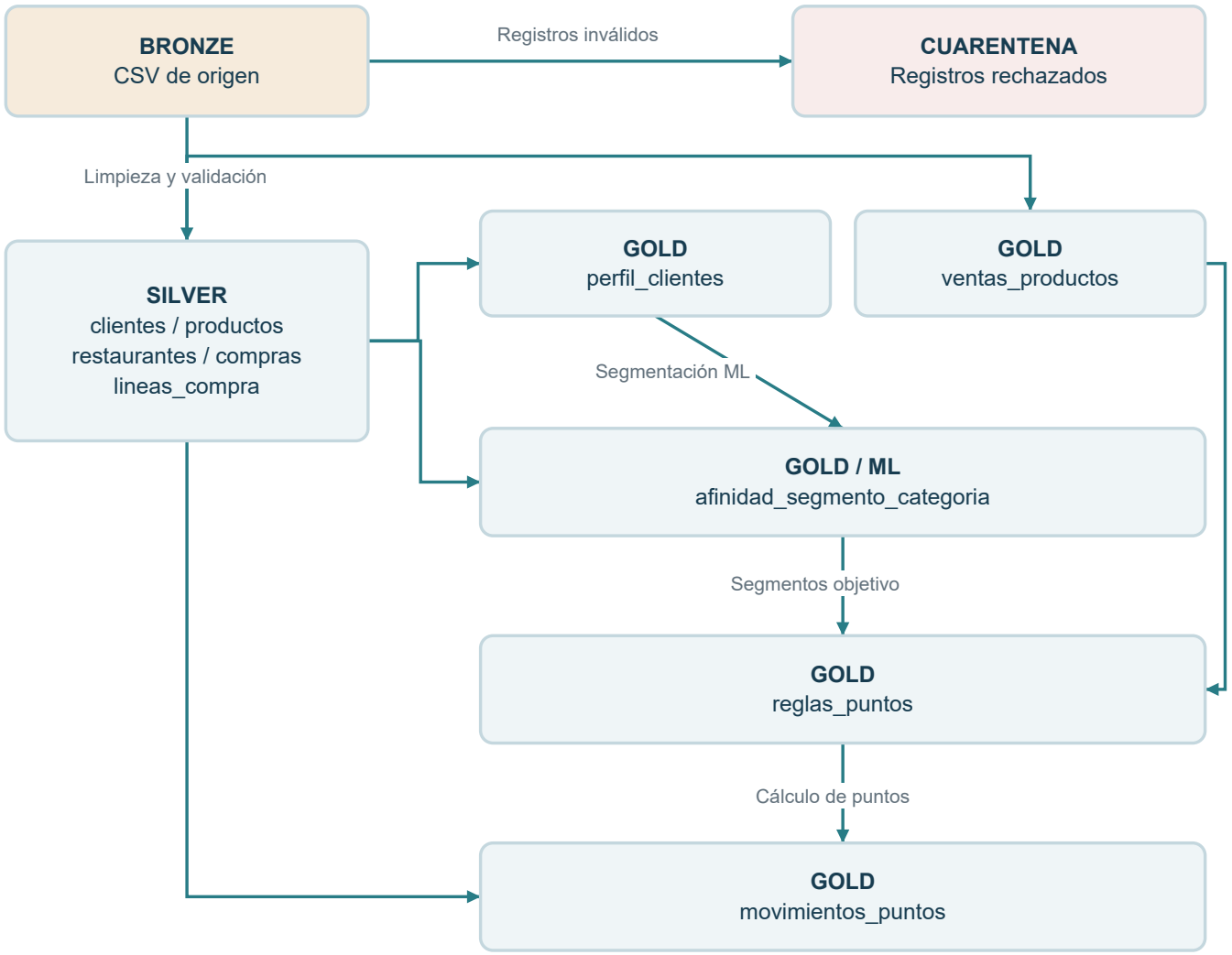
La arquitectura Medallion es una forma de **organizar los datos por niveles de calidad**, no una tecnología específica. Por eso puede implementarse localmente: cada capa es una carpeta y **DuckDB** funciona como motor de consulta. DuckDB es una base de datos analítica que no necesita servidor ni instalación compleja, lee directamente archivos CSV y Parquet, y permite hacer las transformaciones en SQL, de forma similar a un Data Warehouse.



Componente	Herramienta	Función
Datos de origen	Archivos CSV	Simulan las transacciones de McDonald's, un archivo por tabla
Capas de datos	Carpetas con archivos CSV y Parquet	Bronze, Silver y Gold, más una carpeta de cuarentena
Motor de consulta	DuckDB	Lee, transforma y agrega los datos con SQL
Pipeline	Notebooks de Jupyter numerados	Ejecutados en orden, cumplen el papel del orquestador
Machine Learning	scikit-learn	Segmentación de clientes con K-means
Resultados	Tablas y gráficas en el notebook final	Comparan emisión fija/dinámica y explican productos y segmentos

Se usa **Parquet** en Silver y Gold porque, a diferencia del CSV, conserva los tipos de dato (fechas, decimales, enteros).

3 Arquitectura de datos (Medallion)



- **Bronze:** los CSV de origen. Sirve de respaldo para reprocesar si algo falla.
- **Silver:** datos limpios. Se eliminan duplicados, montos negativos, campos vacíos y referencias a productos o clientes inexistentes, y se corrigen los tipos de dato. Los registros rechazados se guardan en `cuarentena` para poder revisarlos.
- **Gold:** tablas para el negocio y el modelo.

El período de datos se divide en dos ventanas: los **meses 1 y 2** sirven para analizar a los clientes y productos y definir las reglas, y el **mes 3** es donde se aplican. Así reservamos un período distinto para comparar la aplicación de las reglas.

Referencias y repositorio

- Databricks. What is a medallion architecture?.
- Términos y condiciones de MiMcDonald's Guatemala.
- Repositorio de GitHub.